

OpenMHP: An Open Protocol for Safe, Context-Efficient Agent Control of Laboratory and Industrial Instruments

Kushal Sinha

kushal.sinha@kortana.io

[TODO: affiliation, ORCID, any co-authors]

October 7, 2026

Abstract

Large language model agents have become competent at using software tools, largely through the Model Context Protocol (MCP), which standardises how an agent discovers and invokes a tool. Physical laboratory and industrial equipment resists the same treatment for three reasons: an operation may run for minutes or hours rather than returning immediately; the instrument emits telemetry, progress and safety events that the agent must observe while the work proceeds; and an incorrect request can destroy a sample or damage hardware, so correctness cannot be delegated to the model’s judgement. We present OpenMHP (Open Model Hardware Protocol), an open protocol that extends the MCP interaction model to physical devices. OpenMHP contributes four design decisions. First, physical operations are modelled as *durable jobs* acquired under a *session lease*, where the lease doubles as a dead-man’s switch: a device declaring a watchdog fails safe if the controlling process goes silent. Second, every request passes six *ordered safety gates* evaluated on the device side of the network, so the enforced envelope is independent of the model’s context or intent. Third, each device is distributed as a *device package* structured like an Agent Skill, enabling progressive disclosure of operating knowledge; on a simulated 2,000-device laboratory, locating and operating one instrument costs 1,270 tokens against 1,224,382 tokens for naive descriptor loading, a reduction to 0.10%. Fourth, the protocol is explicitly a *bridging* layer: adapters expose OPC UA, ROS 2, SiLA 2, MADSci, PyLabRobot and MQTT devices without replacing existing control stacks. We describe the protocol, its trust boundary, a reference implementation in Python, and its conformance suite.

Keywords: laboratory automation; self-driving laboratories; AI agents; large language models; Model Context Protocol; robot safety; scientific instrumentation.

1 Introduction

Autonomous and semi-autonomous experimentation has moved from demonstration to practice. Systems such as Coscientist [10] and the A-Lab [11] have shown that a language model, given actuation and feedback, can plan and execute real experimental campaigns. In parallel, evolutionary and search-based agents such as AlphaEvolve [13] have demonstrated that agents can conduct genuine discovery when the feedback loop is fast and the object of study is symbolic. The corresponding loop in the physical sciences is far slower, and much of its latency is not scientific but

infrastructural: every instrument exposes a different programming interface, and connecting an agent to each one is bespoke work.

The software world addressed the analogous fragmentation with the Model Context Protocol (MCP) [1], which gives an agent one standard way to discover and call a tool, and a tool one standard way to describe itself. MCP’s abstraction is a request that returns a response. That abstraction is a poor fit for physical work, in three specific ways.

Duration. A thermocycling protocol runs for two hours; a liquid-handling routine for twenty minutes; an incubation overnight. A tool call that blocks for the duration is not viable, and one that returns immediately discards the result.

Observation. While the instrument works it produces readings, progress, operator prompts and fault conditions. An agent that can only poll either wastes context or misses events. The natural shape is a push stream, which a request/response tool call cannot express.

Consequence. A malformed software call returns an error. A malformed physical call can over-heat a reagent, crash a robot arm into a plate, or spoil an irreplaceable sample. Safety therefore cannot be a property of the prompt, the model, or the tool description, all of which are inside the agent’s trust domain and all of which the model can, in principle, talk its way around.

OpenMHP is our response. It retains MCP’s wire format, capability negotiation and host/client/server split, so that an OpenMHP deployment is usable from any MCP-speaking harness today, and adds exactly what the three gaps above require. This paper makes the following contributions:

1. A protocol design (Section 4) in which physical work is a durable job held under a session lease, with pushed notifications, six primitives, and a device descriptor that doubles as the instrument’s manual.
2. A device-side safety model (Section 5) built on six ordered gates and an explicit trust boundary in which the host is trusted and the model is not, with fail-closed semantics for operations of unknown outcome.
3. A context-scaling mechanism (Section 6) combining descriptor detail tiers, a live-pinging directory, a constant tool surface and programmatic runs, measured at 0.10% of naive context cost on a 2,000-device laboratory.
4. An open reference implementation and conformance suite (Sections 8–9), including adapters for six existing control ecosystems.

2 Background and Related Work

2.1 Agent-facing protocols

MCP [1] standardises tool discovery and invocation for language-model applications over JSON-RPC 2.0 [4]. Its first year surfaced a scaling problem directly relevant to laboratories: loading every tool definition up front consumed roughly 55,000 tokens for five ordinary servers, and selection accuracy degraded as the list grew [2]. The reported mitigations — deferred tool search, programmatic tool calling, and usage examples embedded in definitions — inform Section 6. Agent Skills [3] established a complementary convention for packaging procedural knowledge that is loaded only when a task requires it; OpenMHP applies that convention to instruments.

2.2 Laboratory and industrial control standards

SiLA 2 [5] provides a gRPC-based standard for laboratory instrument interfaces with strong typing and feature definitions. OPC UA [6] is the dominant information model in industrial automation. ROS 2 [7] supplies the middleware for most research robotics. MADSci [9] and PyLabRobot [8] provide orchestration and hardware-agnostic liquid handling respectively. These layers solve machine-to-machine integration well, and OpenMHP does not attempt to replace them. What none of them provides is an *agent-facing* surface: a natural-language account of what an instrument is for, when not to use it, and which operations require a human, together with a context budget suitable for a model. OpenMHP supplies that surface and delegates actuation downward (Section 7).

2.3 Protocol and procedure description languages

Autoprotocol and LabOP [14] describe experimental procedures in machine-readable form, addressing *what* to run. OpenMHP is complementary and addresses *how an agent safely drives a specific instrument*, including the run-time envelope that a procedure description cannot enforce.

2.4 Self-driving laboratories

ChemOS [12] and successors orchestrate closed-loop experimentation, typically within one integrated facility and one software stack. OpenMHP targets the heterogeneous case: an ordinary laboratory with instruments from many vendors, several control stacks, and no single orchestrator, where the agent is supplied by the user rather than the facility.

3 Design Goals and Trust Model

OpenMHP is specified against five rules, in priority order.

1. **Safety is enforced by the driver, never by the agent.** Limits, interlocks and approval levels live in the descriptor and are checked on the device side of the wire.
2. **One descriptor is the whole manual.** Tacit operating knowledge is carried in natural-language **notes** fields alongside machine-readable types and limits.
3. **Six primitives, no more.** A device that speaks **describe/signals/settings/actions/safety/methods** can be operated by any OpenMHP host.
4. **Bridge, do not replace.** Existing control layers become OpenMHP devices through thin adapters.
5. **The agent’s context is a protected resource.** A laboratory with two thousand devices costs approximately what a laboratory with two costs.

Trust boundary. The central assumption is that *the host is trusted and the model is not*. The host is the agent runtime; it holds clients, presents confirmation prompts to people, and is assumed to execute faithfully. The model it hosts is treated as an untrusted source of requests. Concretely, the flag **approved: true** is the host’s assertion that a person confirmed one exact request; a reference bridge strips and ignores any **approved** value originating from the model. Package code, recipes and orchestration scripts are executed by the host and therefore sit inside the trusted

domain: a deployment in which the model can edit a device package or run unsandboxed scripts has moved the enforcement boundary, and we state this explicitly rather than claiming unconditional guarantees (Section 10).

4 Protocol Design

4.1 Architecture

An OpenMHP deployment comprises a *host* (the agent runtime), one *client* per device, and one *server* per device. A server owns exactly one logical device, serves its descriptor, enforces the safety envelope, translates primitives into vendor commands and runs jobs. A server may equally be an *adapter* wrapping another control layer; from the host’s perspective the two are indistinguishable. Messages are JSON-RPC 2.0 over either stdio or HTTP with Server-Sent Events.

4.2 Primitives

The protocol defines six primitives: **device/describe** (identity and capability, see Section 6); **signals/read** (sensed values); **settings/write** (setpoints, limit-checked); **actions/invoke** (long-running work, returning a job); **safety/*** (**estop**, **reset**); and **methods/*** (named, versioned parameter sets filed per project or compound, for instruments such as chromatographs that operate by named method rather than by free parameters). Session management (**session/acquire**, **renew**, **release**) and job control (**jobs/status**, **pause**, **resume**, **cancel**) support these.

4.3 Jobs

actions/invoke returns immediately with a job identifier rather than a result. Jobs progress through **queued** → **running** → **done** | **failed** | **cancelled**, with an additional **waiting_operator** state for instruments whose steps are performed by a person. The agent may disconnect and reconnect without losing the state of the physical process. A failed job carries **outcome**: **"unknown"** and places the device in **fault** until a verified **safety/reset**, on the principle that a partially completed physical action must not be assumed benign.

4.4 Leases and the watchdog

session/acquire{ttl} grants the calling session exclusive write and invoke rights until the TTL lapses without renewal or the session releases them. Every client has its own session identifier, so independent clients never share a lease. Reads, **ping**, **describe**, **jobs/status**, **jobs/pause**, **jobs/cancel** and **safety/estop** are deliberately never blocked by a lease, so that observation and stopping remain always available.

The lease additionally serves as a dead-man’s switch. If a descriptor declares **safety.watchdog_s**, the watchdog is armed whenever a session holds the lease; any request from the holder renews it. Should the holder fall silent for longer than the declared window, or should its lease expire without release, the server must fail safe: enter **estop**, cancel active jobs, run the stop hook and drop the lease. Recovery requires a verified reset. This converts the liveness of the controlling process into a safety property, which we regard as the single most important difference between driving an instrument and calling a software tool.

4.5 Notifications

Servers push JSON-RPC notifications for signal updates, job start, progress, completion, pause/resume/cancel, e-stop, fault, operator instructions, and method changes. A supervising host consumes these without being asked; the reference bridge subscribes to every device it opens, records events against the owning run, and writes notable ones to an append-only run log. No part of this path polls the instrument.

4.6 The device package

A device is distributed as a folder, deliberately shaped like an Agent Skill:

```
devices/hotplate-01/
  DEVICE.md          # YAML frontmatter (card) + operating instructions in prose
  descriptor.yaml    # signals, settings with limits, actions, interlocks, safety
  driver.py          # a Driver subclass, a BoundDriver, or an adapter
  sim.py             # optional simulated twin
  references/        # SOPs, manual excerpts
  scripts/           # tested orchestration scripts
  methods/           # optional saved methods per project
```

The separation matters: `DEVICE.md` carries what a new colleague would be told, in the owner's own words, while `descriptor.yaml` carries what a machine must enforce. The same artefact therefore serves both the model's understanding and the driver's gate checks, and the two cannot drift apart silently.

5 Safety Model

5.1 Six ordered gates

On every `settings/write` and `actions/invoke`, the server evaluates, in this fixed order:

1. **State** — `estop` yields `EStopActive`; `fault` yields `DeviceFault`.
2. **Approval** — `forbid` yields `Forbidden`; `confirm` without `approved: true` yields `ApprovalRequired`.
3. **Interlocks** — each named signal is read live and must be exactly `true`; false, unreadable or stale yields `InterlockOpen`.
4. **Values** — type, then declared limits; for actions, declared, required and bounded parameters, then the driver's whole-request `validate_params`.
5. **Lease** — a lease held by another session, or a watchdog device operated without one, yields `NotLeased`.
6. **Busy** — a non-concurrent action while a job runs yields `DeviceBusy`.

Vendor code executes only after all six pass. The order is normative because it determines which refusal an agent sees first, and therefore what it learns from a refusal.

5.2 Human approval and elicitation

Approval level **confirm** is the junction between autonomy and oversight. The sequence is: the model requests the action; the host receives **ApprovalRequired**; the host presents the device, the action, its exact parameters and the descriptor’s notes to a person; and only on an explicit affirmative does the host resend the identical request with **approved: true**. The reference bridge implements this through MCP elicitation. Where a harness offers no elicitation channel, the bridge has no trusted path to a person, and confirm-gated operations are reported unavailable rather than being silently permitted.

5.3 Fail-closed semantics and defence in depth

Any exception inside vendor code during a job, and any write failure occurring after a command may already have reached the instrument, moves the device to **fault** with an unknown outcome and runs the driver’s safe-state hook. There is no automatic recovery. Separately, limits on scalar setpoints are insufficient when a dangerous value is nested inside a structured parameter — for instance a 200 °C step inside an otherwise valid protocol — so the driver’s **validate_params** inspects the whole request, and must do so for dry runs as well, ensuring no step executes before the entire program is known valid.

5.4 Scope

OpenMHP is not a functional-safety system in the IEC 61508 [16] sense. Hard-wired emergency stops, light curtains and PLC safety logic remain the primary protective layer. OpenMHP prevents an agent from *asking* for something unsafe and gives it a standard way to stop everything when it observes a problem. We state this prominently because conflating the two would be the most consequential misreading of this work.

6 Context Scaling

A laboratory has more devices than a workspace has tools, and a device descriptor is longer than a tool schema. Four mechanisms keep the agent’s context approximately independent of laboratory size.

Detail tiers. **device/describe** accepts a **detail** parameter with three tiers: **card** (roughly 40 tokens: identity, location, live state and the *names* of capabilities), **summary** (under 1,000 tokens: the card plus operating instructions plus every capability with type, unit, limits, approval level and interlocks), and **full**. A **select** parameter fetches complete specifications for named items only. The intended path is card → summary → select the two items that will actually be used.

Directory. A directory is a server whose role is finding devices rather than being one. It indexes cards only, ranking with BM25 [15] over identity, class, make, model, location, tags, notes and capability names, so a query such as “*heat a 96-well plate to 95 °C for PCR in bay 12*” resolves without the agent knowing any identifiers. Crucially, *state is never served from the index*: at query time the directory pings top candidates in parallel with a short timeout and fills in live state before applying any state filter, so a stale index cannot cause an agent to address an unreachable instrument.

Constant tool surface. A bridged host sees eleven tools regardless of device count, and never per-device tools. Bridges must not enumerate devices into the tool list at startup.

Programmatic runs. An orchestration script executes against the laboratory and returns only what it prints, capped. A script that takes five hundred readings and reports a mean places one line in the model’s context rather than five hundred numbers. A plan mode rehearses a script with reads real and every mutation a dry run through the same gates, under virtual time and a step budget, producing the ordered steps, those requiring confirmation, those that are long-running, and the first step the driver would refuse.

7 Bridging Existing Ecosystems

All adapters are built on one mechanism. A *binding* pairs an OpenMHP name with a callable into the foreign layer, and `BoundDriver` turns three lists of bindings into a complete driver that inherits every gate, tier, job and notification:

```
from openmhp.adapters import BoundDriver, Signal, Setting, Action
dev = BoundDriver(
    device={"id": "hotplate-01", "class": "hotplate", "notes": "Fume_hood_2."},
    signals=[Signal("plate_temperature", read=lambda: plc.read(0x10), unit="degC")],
    settings=[Setting("target_temperature", write=lambda v: plc.write(0x20, v),
                      limits={"min": 20, "max": 300})],
    actions=[Action("shutdown", run=lambda job, p: plc.write(0x21, 0),
                    approval="confirm")],
    estop=lambda: plc.write(0x21, 0))
```

The reference implementation ships adapters for OPC UA, ROS 2, SiLA 2, MADSci, Py-LabRobot and MQTT on this one mechanism. The practical consequence is that the safety envelope is added *above* a control layer that may itself provide none, without modifying it.

8 Reference Implementation

`openmhp` is a Python implementation of the specification, comprising approximately 7,300 lines across the driver base class, package loader, directory, stdio and HTTP+SSE transports, client SDK, command-line interface, MCP bridge, the six adapters, three bundled Agent Skills, simulated devices and the scale demonstration. PyYAML is the only runtime dependency of the core. Writing a new driver requires a descriptor and three hooks (`on_read`, `on_write`, `on_invoke`), with optional `validate_params` and `on_fault`.

Four conformance levels are defined: **Core** (describe with all tiers, read, limited write, error codes), **Actions** (jobs, interlocks, approval gates, methods), **Safe** (non-blocking e-stop, verified reset, watchdog, fail-closed state machine, per-session leases) and **Directory** (the server role). Devices carrying stored energy or capable of motion should not be exposed below Safe.

9 Evaluation

9.1 Context cost at scale

The reference `examples/scale_demo.py` constructs 2,000 simulated devices across 8 classes and 40 bays, and measures the tokens entering an agent’s context for a representative task: find a suitable instrument, understand it, and operate it.

Strategy	Tokens in agent context
Every descriptor loaded up front	1,224,382
Find (5 cards) + summary of chosen device + full spec of the 2 items used	1,270

Table 1: Context cost on a simulated 2,000-device laboratory. The progressive path costs 0.10% of naive loading, with retrieval well under one millisecond. The device is then operated through exactly the same primitives as in a two-device laboratory.

9.2 Conformance and behavioural testing

Conformance in OpenMHP is behavioural rather than structural. The reference suite comprises 53 tests across approximately 1,800 lines, exercising typed limits and limit violations, lease ownership and contention, watchdog expiry under induced silence, rejected and forged approvals, interlock enforcement, blocked I/O, fail-closed fault latching on backend aborts, plan-mode isolation from real actuation, and the method lifecycle. Adapters are tested against injected fakes of each vendor client library, verifying that the gate behaviour is inherited correctly through `BoundDriver` rather than reimplemented per ecosystem.

9.3 Deployment on physical instruments

[**TODO: This subsection must be written from your real hardware runs before submission; do not submit the paper without it. Reviewers will expect, at minimum: (i) which instruments, by make and model, and which conformance level each was exposed at; (ii) which integration route each used — native driver, serial ASCII, or which adapter; (iii) how long each ran and how many jobs completed; (iv) at least one instance of a gate firing against a real unsafe request, with the refusal; (v) at least one watchdog or e-stop event observed on real hardware, since that is the central safety claim of Section 5; (vi) any failure, surprise or workaround, which reviewers weight heavily in a systems paper; and (vii) whether a domain scientist, rather than the authors, operated it. If a controlled comparison is possible — time or lines of integration code for the same instrument with and without OpenMHP — it would substantially strengthen the paper.**]

10 Limitations and Future Work

We state the boundaries of the current draft explicitly.

- **Not functional safety.** OpenMHP does not replace IEC 61508-class protective systems (Section 5).
- **Enforcement depends on the trust boundary holding.** A model able to edit a device package, or to run arbitrary scripts in an unsandboxed host, can alter what is enforced. Deployments should separate package authoring from operation.
- **Non-idempotent operations.** A request may reach a device even when its response is lost, and servers do not deduplicate. Clients must not retry writes or invocations automatically; they should reconcile by reading state.

- **Unspecified in this draft.** In-band authentication, and authentication for the device HTTP transport; idempotency tokens; multi-device transactions; descriptor signing; a hosted registry; sandboxed script execution; streaming of bulk data; and a machine-readable description of hardware watchdogs and device-side safe states.
- **Evaluation breadth.** [TODO: state honestly how many instruments, of what diversity, and in how many independent laboratories; note explicitly if all deployments are by the authors.]

11 Conclusion

Agents are becoming capable enough that the limiting factor in physical science is increasingly the interface to the instrument rather than the reasoning about the experiment. OpenMHP proposes that this interface should be open, should model physical work as durable leased jobs with pushed observation, should enforce its safety envelope on the device side of the network where the model cannot reach it, and should package instrument knowledge for progressive disclosure so that laboratory size does not consume the agent’s context. The specification, reference implementation and conformance suite are available under Apache 2.0.

Availability

Specification and reference implementation: <https://github.com/kushalsinha/openmhp>. Documentation: <https://openmhp.com>. Released under the Apache License 2.0. [TODO: mint an archival DOI (e.g. via Zenodo) for the exact release this paper describes, and cite it here; journals increasingly require it and it is what others will cite.]

Acknowledgements

[TODO: acknowledgements; funding statement; and a disclosure of AI assistance in the preparation of the software and/or manuscript, which arXiv and most journals now require.]

References

- [1] Anthropic. *Model Context Protocol*. 2024. <https://modelcontextprotocol.io>
- [2] Anthropic. *Advanced tool use*. Engineering blog. [TODO: verify exact title, author and date]
- [3] Anthropic. *Agent Skills*. [TODO: verify citation form]
- [4] JSON-RPC Working Group. *JSON-RPC 2.0 Specification*. 2010. <https://www.jsonrpc.org/specification>
- [5] SiLA Consortium. *SiLA 2 Standard*. <https://sila-standard.org>
- [6] IEC 62541. *OPC Unified Architecture*. International Electrotechnical Commission.
- [7] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, W. Woodall. Robot Operating System 2: Design, architecture, and uses in the wild. *Science Robotics*, 7(66), 2022.
- [8] R. P. Wierenga et al. PyLabRobot: An open-source, hardware-agnostic interface for liquid-handling robots and accessories. *Device*, 2023. [TODO: verify volume and pages]

- [9] Argonne National Laboratory. *MADSci: Modular Autonomous Discovery for Science*. **[TODO: verify canonical citation]**
- [10] D. A. Boiko, R. MacKnight, B. Kline, G. Gomes. Autonomous chemical research with large language models. *Nature*, 624:570–578, 2023.
- [11] N. J. Szymanski et al. An autonomous laboratory for the accelerated synthesis of novel materials. *Nature*, 624:86–91, 2023.
- [12] L. M. Roch et al. ChemOS: Orchestrating autonomous experimentation. *Science Robotics*, 3(19), 2018.
- [13] Google DeepMind. *AlphaEvolve: A coding agent for scientific and algorithmic discovery*. 2025. **[TODO: verify citation form and date]**
- [14] *Laboratory Open Protocol language (LabOP)*. **[TODO: verify citation; or substitute Autoprotocol]**
- [15] S. Robertson, H. Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.
- [16] IEC 61508. *Functional safety of electrical/electronic/programmable electronic safety-related systems*. International Electrotechnical Commission.